

2025 Retrospective

This idea came together sometime around 2012 during one of my Netflix-on-a-beanbag binges. It might have been *Doctor Who* playing in the background, but the basic goal was simple: write a specification and generate a working integration from it.

At the time, I was maintaining more than 20 integrations for systems like Jira, Quality Center, Git, Perforce, and several others. My friend and colleague Josh Gough liked to call them "outergrations" because they were substantial pieces of software with their own processes, deployment requirements, and operational footprint. Most were built on the .NET ServiceHost framework, though a few were Java-based and there were some miscellaneous plugin integrations mixed in as well.

They definitely served a purpose. Customers liked them, and in some cases they even dedicated an engineer and a server just to keep them running. Good times.

Back then, if a customer wanted an integration, they typically paid the VersionOne Services team to build it. I started thinking there had to be a more structured and repeatable way to create integrations through a defined specification language. You could think of it as Integration as Code (IaC), before that term became attached to infrastructure.

The idea was that a specification could be compiled into a running executable or plugin capable of managing a point-to-point integration between systems. Longer term, I imagined it evolving toward a single integration platform, similar in spirit to Tasktop, while still allowing teams to extend behavior by dropping in a tiny bit of JSON and writing relatively small integration specifications instead of building a big top heavy relative mammoth.

Looking back, it was probably a little ambitious, but it felt like a practical way to tame the growing collection of custom integrations we were supporting.

INTEGEN

A Declarative Integration Generation Environment

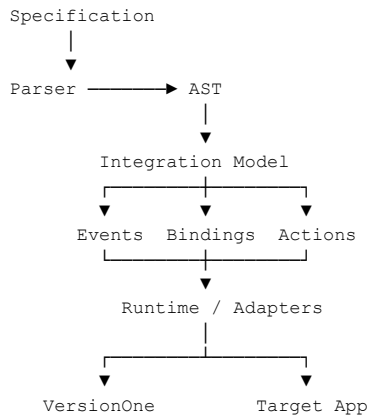
Design Document May 2012

“When something happens, do something.”

1. Executive Summary

Integen was a proposed code-development utility for generating VersionOne integrations. The problem was straightforward: creating integrations between VersionOne and third-party applications required substantial development experience. The design introduced an English-like declarative language so a user could describe the integration while the engine handled the underlying integration abstractions.

The core model was deliberately simple: when something happens, do something; do something repeatedly; synchronize something; or copy something. Transactions provided units of work, while events, actions, bindings, timers, and adapters provided the machinery underneath.



The design also included a code-generation path: Smidgen language → AST → templates / C# fluent-interface code → generated Visual Studio project → executable integration.

2. Design Goal

The goal was not to eliminate programming. It was to create a reusable programming model for integrations so that the author could express intent at a higher level. A technical user could work directly with the language; a less technical user could use the English-like form or UI. The same underlying engine would translate that description into executable integration code.

3. The Integration Model

An integration is a set of instructions organized into transactions or execution units. The original documents call these units paragraphs in the Smidgen model: each paragraph contains one or more sentences, and the paragraph can be placed into an execution queue.

4. Language and Syntax

The language was intended to read like a description of the integration rather than like conventional application code.

```
Integration "V1toTwitter"

connection[V1] = Authenticate("http://path", username, password)
authenticate[Twitter] = Authenticate("http://path", username, password)

when: V1(Creates test in ProjectName)

transaction {
  do: Actions(Create test in BigProject)
  push: MapsSync(
    ('Name', 'fName'),
    ('Description', 'Body'),
    RTOL
  )
  set:
}
```

The key keywords had explicit roles: **when** is the main condition, **do** is the resulting action, **push** identifies fields to propagate, **set** establishes fields to set, and **Cycle** represents how long to wait before an action.

5. A More Human-Oriented Form

The design also experimented with a less technical syntax:

```
Create VersionOne Server with url "http://..." with username "foo" and password "blah".
Create AppFuzz Server with url "http://..." with username "foo" and password "blah".

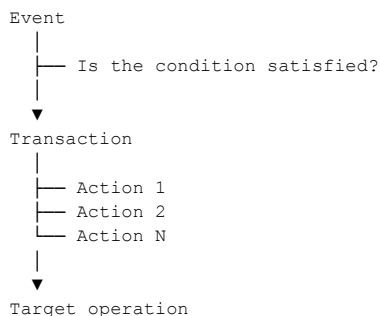
When a test is created in folder "Left.Root.My folder",
duplicate in "Right.Test Folder"

create sync between VersionOne and Target
End
```

The intent was that the author should be able to describe the integration in terms of the objects and behavior being connected, while the framework supplied the implementation details.

6. Events and Transactions

An event identifies something the engine should watch for. The design included asset creation, update, deletion, application online/offline state, and timer events.



7. Action Composition

An event was not limited to one operation. The design treated an event action as a series of items that need to be done.

```
OnEvent.Created.VersionOne.Test:
    [Event(),
      Action.QCAddTest(),
      Action.QCSomething()]
```

The framework therefore decomposed integration behavior into reusable operations. The proposed operation pool included CRUD operations for both VersionOne and the target application.

VersionOne		Target
Create Asset	→	Create Asset
Read Asset	←	Read Asset
Update Asset	←	Update Asset
Delete Asset	←	Delete Asset

8. Polling and Event Simulation

Because VersionOne did not provide the desired event mechanism, the design proposed simulating eventing through gentle polling. The important architectural point was that polling lived beneath the event abstraction: the integration author could describe an event without needing to know whether the implementation used polling or another mechanism.

9. Data Binding

A binding describes the relationship between attributes on the two sides of an integration. VersionOne was normally the left/root side and the target application the right side.

```
{ "Description": "Description" }
{ "Description": "Result" }
```

The design recognized that field names might differ and that types might differ. It therefore considered transformations such as casts.

```
{
  "Description": {
    "Result": "Cast(int)"
  }
}
```

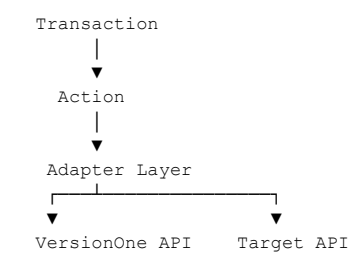
Bindings were not merely field-name pairs. Transactions supplied the rule for when a binding should be applied. For example: when Right.Status is updated, copy Right.Status to Left.Status.

10. Direction

Direction was explicit. RTOL and LTOR described which side supplied the source data. The design assumed VersionOne on the left and the target on the right.

11. The Adapter Boundary

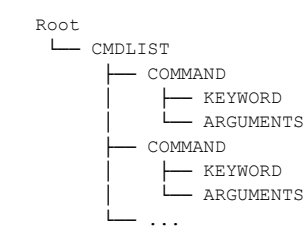
The design deliberately separates the integration model from vendor-specific API mechanics. An action could contain adapters passing data back and forth between applications.



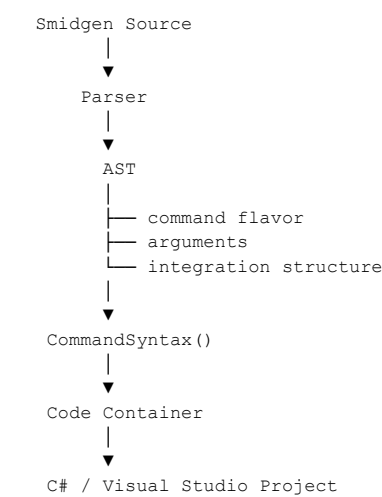
This boundary is important because it gives the language a stable model even when the implementation details of a vendor API change.

12. AST and Code Generation

The compiler design was built around a predictable abstract syntax tree. The stated goal was to make commands have a similar structure and reduce recursively defined grammar rules.



The source describes a four-level tree in which command nodes appear at one level and their arguments at the next. Command flavor could then dispatch to `CommandSyntax()`, while `ArgumentDfn()` described argument count and kinds.



The generated result was intended to be a normal Visual Studio project that could be built, configured, and debugged.

13. Worked Example: AppFuzz → VersionOne

One use case is a defect-tracking integration. A project manager wants new AppFuzz defects created in VersionOne. Four fields are identified for the initial synchronization: Issue Name, Description, Owner, and Status.

```
Connect(V1, URL);
Connect(AF, URL);

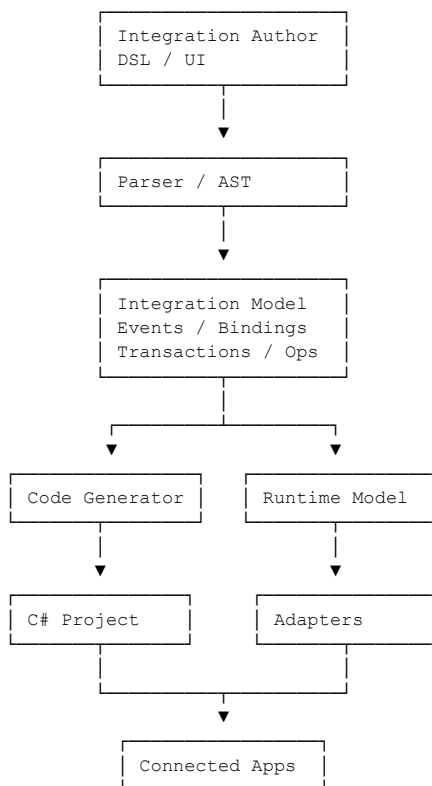
SetLink(Defect, V1, title,
        AF, Title);

SetLink(Defect, V1, description,
        AF, Description);

PushData(DEFFECT, CREATE, AF, V1);

Execute;
```

14. End-to-End Design



15. Design Decisions

Declarative over imperative. Describe what should happen instead of forcing the integration author to implement every API interaction.

Events as an abstraction. The language expresses an event even when the underlying platform requires polling.

Transactions as units of work. A trigger can produce a predictable sequence of operations.

Bindings separate data from behavior. Field relationships can be represented independently from the rule that applies them.

Adapters isolate vendor APIs. Vendor-specific mechanics belong behind an integration boundary.

Predictable AST. A regular tree makes command dispatch and code generation easier to reason about.

Generated code remains visible. The proposed output is a real Visual Studio project rather than an opaque runtime artifact.

16. Framework Model

The final framework concept described a code container divided into compartments: connections, a process model, an events pool, an operations pool, logging, endpoint negotiation, and code targets.

INTEGEN FRAMEWORK
Connection
Process Model: pub/sub poll webservice
Events Pool
Operations Pool
Logging
Endpoint Negotiation
Code Target: C# Java JavaScript / Node

17. Specification Summary

Concept	Purpose
Integration	Defines the connected systems and overall integration.
Connection	Establishes access to a system.
Event / When	Defines what causes a transaction to execute.
Transaction	Defines a unit of work.
Action / Do	Performs an operation.
Binding / Push / Sync	Defines how data moves between fields.
Direction	Defines the source side of a mapping.
Cycle / Timer	Controls repeated or scheduled behavior.
Adapter	Encapsulates vendor-specific API interaction.
AST	Represents the parsed specification.
Code Generator	Transforms the model into executable source/project structure.

18. Conclusion

Integen was an attempt to turn integration development into a language and framework rather than a collection of one-off programs. Its central abstraction was simple: describe an event, define the transaction, identify the data to move, and let the framework handle the mechanics.

The design connected several layers that are normally treated separately: a domain language for expressing integration intent; an AST for representing that intent; transactions and events for execution; bindings for data movement; adapters for vendor APIs; and code generation for producing a usable application. The original documents repeatedly return to the same idea: the integration author should describe the behavior, while the engine supplies the machinery.

That combination—abstraction, explicit models, predictable execution, and generated code is the core of the Integen design.